

VQE

```
In [1]: from qiskit_algorithms import NumPyMinimumEigensolver, SamplingVQE
from qiskit_algorithms.optimizers import COBYLA
from qiskit_algorithms.utils import algorithm_globals
from qiskit.primitives import Sampler
from qiskit_optimization.algorithms import MinimumEigenOptimizer
from qiskit_optimization.translators import from_docplex_mp
from qiskit.circuit.library import TwoLocal

import pylab
from docplex.mp.model import Model

import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: # Model
qp_model = Model("Model for VQE")

x1, x2, x3 = qp_model.binary_var_list([1,2,3], name="x")

x4 = qp_model.integer_var(0, 10, name="x4")
x5 = qp_model.integer_var(0, 10, name="x5")

qp_model.maximize(x1**2 + 2*x1 + 1*x2 + -7*x3 + 4*x3 + 2*x4 + 9*x5 - x1*2

qp_model.add_constraint( x4 + x5 >= 15)

# case to
qp = from_docplex_mp(qp_model)
print(qp.prettyprint())
print(qp_model.is_quadratic())
```

Problem name: Model for VQE

Maximize

$$x_1^2 + 5x_4 - x_5 + x_2 - 3x_3$$

Subject to

Linear constraints (1)

$$x_4 + x_5 \geq 15 \quad 'c0'$$

Integer variables (2)

$$0 \leq x_4 \leq 10$$

$$0 \leq x_5 \leq 10$$

Binary variables (3)

$$x_1 \ x_2 \ x_3$$

True

```
In [3]: ansatz = TwoLocal(5, "ry", "cz", reps=3, entanglement="full")
```

```
In [4]: ev_counts = []
ev_values = []

def store_intermediate_result(eval_count, parameters, mean, std):
    ev_counts.append(eval_count)
    ev_values.append(mean)
```

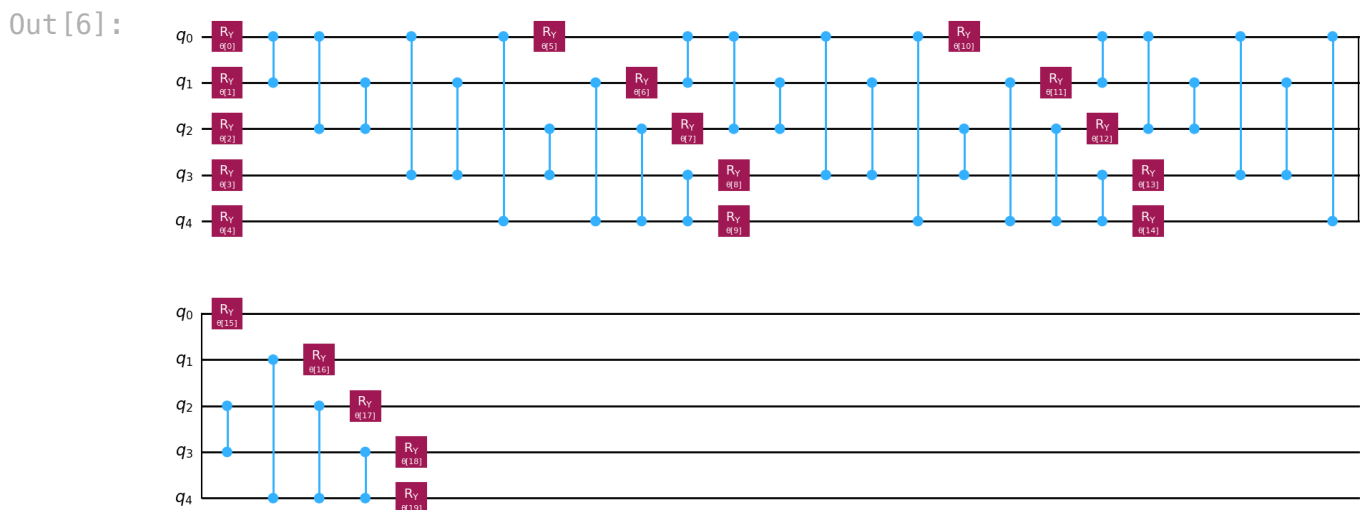
```
In [5]: algorithm_globals.random_seed = 1234

cobyla = COBYLA()
cobyla.set_options(maxiter=100)

svqe_mes = SamplingVQE(
    sampler=Sampler(options={"shots": 1024}),
    ansatz=ansatz,
    optimizer=cobyla,
    callback=store_intermediate_result,
)

svqe = MinimumEigenOptimizer(svqe_mes)
```

```
In [6]: svqe_mes.ansatz.decompose().draw(output="mpl")
```



```
In [7]: result = svqe.solve(qp)
```

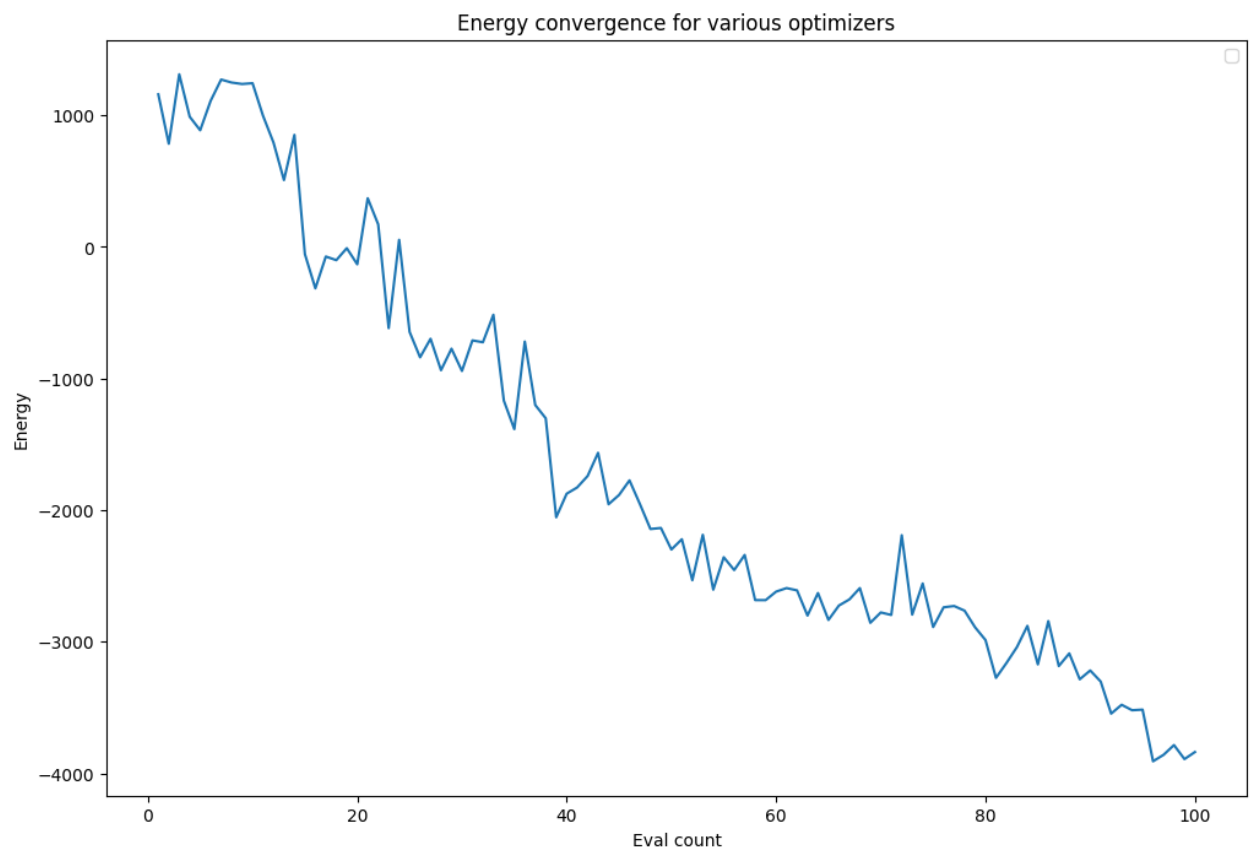
```
In [8]: print(result.prettyprint())
```

```
objective function value: 46.0
variable values: x_1=1.0, x_2=1.0, x_3=0.0, x_4=10.0, x_5=6.0
status: SUCCESS
```

```
In [9]: pylab.rcParams["figure.figsize"] = (12, 8)

pylab.plot(ev_counts, ev_values)
pylab.xlabel("Eval count")
pylab.ylabel("Energy")
pylab.title("Energy convergence for various optimizers")
pylab.legend(loc="upper right")
```

```
Out[9]: <matplotlib.legend.Legend at 0x1684ba540>
```



```
In [10]: # solve classically as reference
opt_result = MinimumEigenOptimizer(NumPyMinimumEigensolver()).solve(qp)
print(opt_result.prettyprint())
```

objective function value: 47.0

variable values: x_1=1.0, x_2=1.0, x_3=0.0, x_4=10.0, x_5=5.0

status: SUCCESS